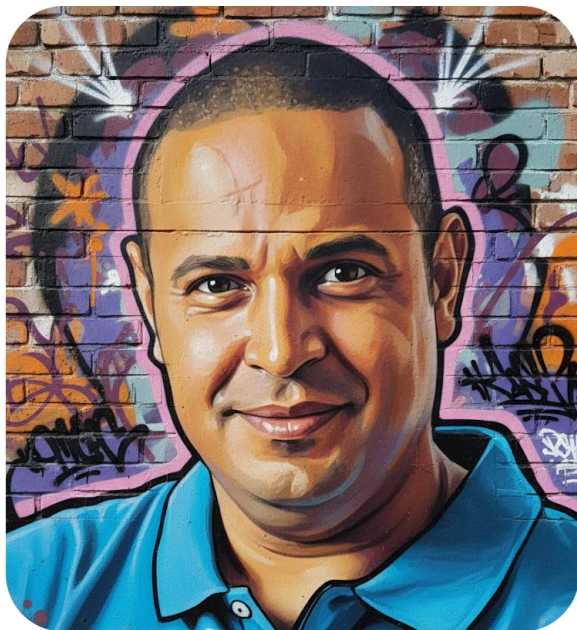


DEEP DIVE INTO THE SELECT STATEMENT

JESÚS ESPINO
SOFTWARE ENGINEER
@ VictoriaMetrics



ABOUT ME



- **SOFTWARE ENGINEER AT**  **VictoriaMetrics**
- **OPEN SOURCE ENTHUSIAST (AND CONTRIBUTOR)**
- **DEEP KNOWLEDGE FAN**
- **BLOGGER: [HTTPS://INTERNALS-FOR-INTERNS.COM/](https://internals-for-interns.com/)**
- **BOOK AUTHOR**

Internals for Interns

About Me Series Tags

Article Series

Explore in-depth series covering various technical topics



Understanding the Go Compiler

8 articles · Started October 2025

Articles in this series:

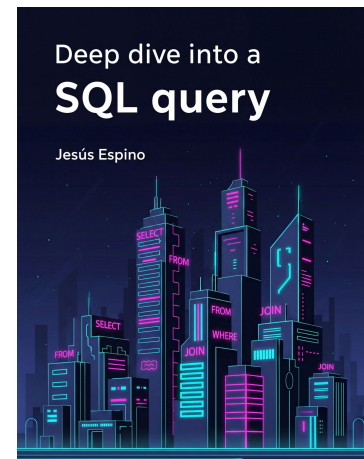
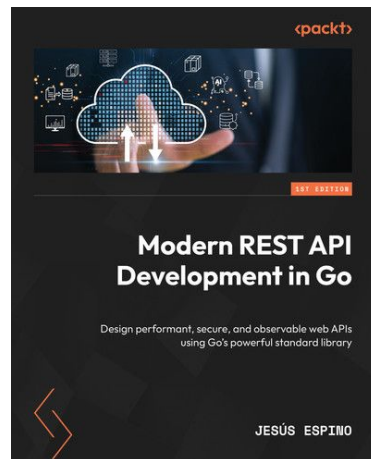
The Scanner
The Parser
The Type Checker
+ 5 more articles

[View Series →](#)



The SQL Query Roadtrip

7 articles · Started October 2025



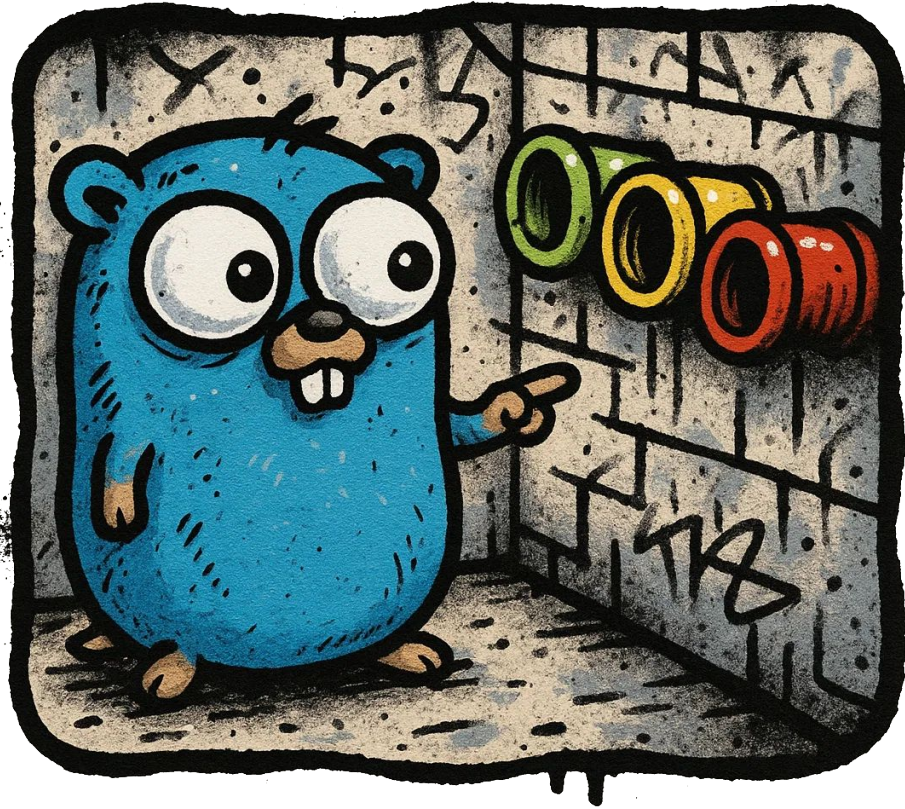
DISCLAIMER



DISCLAIMER



SELECT



SELECT

```
select {  
  case msg1 := <-ch1:  
    fmt.Println("Received:", msg1)  
  case msg2 := <-ch2:  
    fmt.Println("Received:", msg2)  
}
```

```
select {  
  case msg := <-ch:  
    fmt.Println("Received:", msg)  
  default:  
    fmt.Println("No message received")  
}
```

```
select {  
  case ch <- "hello":  
    fmt.Println("Sent message")  
  default:  
    fmt.Println("Channel full, couldn't send")  
}
```

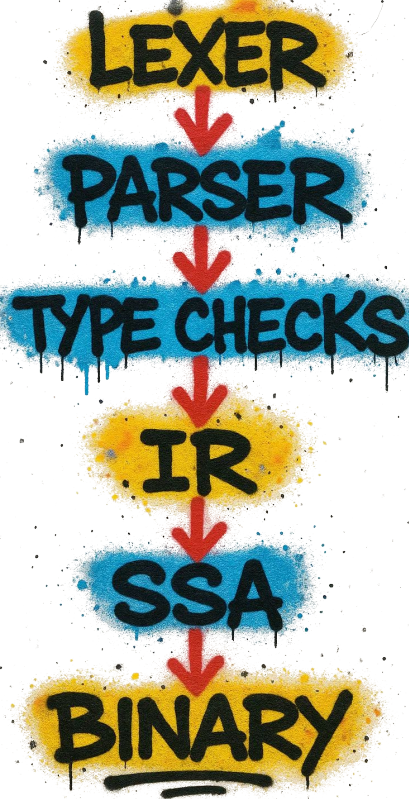

WHY SELECT?



SELECT AND THE COMPILER



THE COMPILER PROCESS



TYPE CHECKS

- ***CHECK FOR MULTIPLE DEFAULTS***
- ***CHECK CASES TYPES***
 - ***READ CHANNEL***
 - ***WRITE CHANNEL***

TYPE CHECKS



```
func (check *Checker) multipleSelectDefaults(list []*syntax.CommClause) {  
    var first *syntax.CommClause  
    for _, c := range list {  
        if c.Comm == nil {  
            if first != nil {  
                check.errorf(c, DuplicateDefault, "multiple defaults (first at %s)", first.Pos())  
                // TODO(gri) probably ok to bail out after first error (and simplify this code)  
            } else {  
                first = c  
            }  
        }  
    }  
}
```

src/cmd/compile/internal/types2/stmt.go

TYPE CHECKS



```
func (check *Checker) multipleSelectDefaults(list []*syntax.CommClause) {  
    var first *syntax.CommClause  
    for _, c := range list {  
        if c.Comm == nil {  
            if first != nil {  
                check.errorf(c, DuplicateDefault, "multiple defaults (first at %s)", first.Pos())  
                // TODO(gri) probably ok to bail out after first error (and simplify this code)  
            } else {  
                first = c  
            }  
        }  
    }  
}
```

src/cmd/compile/internal/types2/stmt.go

TYPE CHECKS



```
func (check *Checker) multipleSelectDefaults(list []*syntax.CommClause) {  
    var first *syntax.CommClause  
    for _, c := range list {  
        if c.Comm == nil {  
            if first != nil {  
                check.errorf(c, DuplicateDefault, "multiple defaults (first at %s)", first.Pos())  
                // TODO(gri) probably ok to bail out after first error (and simplify this code)  
            } else {  
                first = c  
            }  
        }  
    }  
}
```

src/cmd/compile/internal/types2/stmt.go

TYPE CHECKS



```
func (check *Checker) multipleSelectDefaults(list []*syntax.CommClause) {  
    var first *syntax.CommClause  
    for _, c := range list {  
        if c.Comm == nil {  
            if first != nil {  
                check.errorf(c, DuplicateDefault, "multiple defaults (first at %s)", first.Pos())  
                // TODO(gri) probably ok to bail out after first error (and simplify this code)  
            } else {  
                first = c  
            }  
        }  
    }  
}
```

src/cmd/compile/internal/types2/stmt.go

TYPE CHECKS



```
for _, clause := range s.Body {
    if clause == nil {
        continue // error reported before
    }

    // clause.Comm must be a SendStmt, RecvStmt, or default case
    valid := false
    var rhs syntax.Expr // rhs of RecvStmt, or nil
    switch s := clause.Comm.(type) {
    case nil, *syntax.SendStmt:
        valid = true
    case *syntax.AssignStmt:
        if _, ok := s.Rhs.(*syntax.ListExpr); !ok {
            rhs = s.Rhs
        }
    case *syntax.ExprStmt:
        rhs = s.X
    }
    ...
}
```

src/cmd/compile/internal/types2/stmt.go

TYPE CHECKS



```
for _, clause := range s.Body {
    if clause == nil {
        continue // error reported before
    }

    // clause.Comm must be a SendStmt, RecvStmt, or default case
    valid := false
    var rhs syntax.Expr // rhs of RecvStmt, or nil
    switch s := clause.Comm.(type) {
    case nil, *syntax.SendStmt:
        valid = true
    case *syntax.AssignStmt:
        if _, ok := s.Rhs.(*syntax.ListExpr); !ok {
            rhs = s.Rhs
        }
    case *syntax.ExprStmt:
        rhs = s.X
    }
    ...
}
```

src/cmd/compile/internal/types2/stmt.go

TYPE CHECKS



```
for _, clause := range s.Body {
    if clause == nil {
        continue // error reported before
    }

    // clause.Comm must be a SendStmt, RecvStmt, or default case
    valid := false
    var rhs syntax.Expr // rhs of RecvStmt, or nil
    switch s := clause.Comm.(type) {
    case nil, *syntax.SendStmt:
        valid = true
    case *syntax.AssignStmt:
        if _, ok := s.Rhs.(*syntax.ListExpr); !ok {
            rhs = s.Rhs
        }
    case *syntax.ExprStmt:
        rhs = s.X
    }
    ...
}
```

src/cmd/compile/internal/types2/stmt.go

TYPE CHECKS



```
for _, clause := range s.Body {
    if clause == nil {
        continue // error reported before
    }

    // clause.Comm must be a SendStmt, RecvStmt, or default case
    valid := false
    var rhs syntax.Expr // rhs of RecvStmt, or nil
    switch s := clause.Comm.(type) {
    case nil, *syntax.SendStmt:
        valid = true
    case *syntax.AssignStmt:
        if _, ok := s.Rhs.(*syntax.ListExpr); !ok {
            rhs = s.Rhs
        }
    case *syntax.ExprStmt:
        rhs = s.X
    }
    ...
}
```

src/cmd/compile/internal/types2/stmt.go

TYPE CHECKS



```
for _, clause := range s.Body {  
    ...  
  
    // if present, rhs must be a receive operation  
    if rhs != nil {  
        if x, _ := syntax.Unparen(rhs).(*syntax.Operation); x != nil && x.Y == nil && x.Op == syntax.Recv {  
            valid = true  
        }  
    }  
  
    if !valid {  
        check.error(clause.Comm, InvalidSelectCase, "select case must be send or receive...")  
        continue  
    }  
    ...  
}
```

TYPE CHECKS



```
for _, clause := range s.Body {  
    ...  
  
    // if present, rhs must be a receive operation  
    if rhs != nil {  
        if x, _ := syntax.Unparen(rhs).(*syntax.Operation); x != nil && x.Y == nil && x.Op == syntax.Recv {  
            valid = true  
        }  
    }  
  
    if !valid {  
        check.error(clause.Comm, InvalidSelectCase, "select case must be send or receive...")  
        continue  
    }  
    ...  
}
```

TYPE CHECKS



```
for _, clause := range s.Body {  
    ...  
  
    // if present, rhs must be a receive operation  
    if rhs != nil {  
        if x, _ := syntax.Unparen(rhs).(*syntax.Operation); x != nil && x.Y == nil && x.Op == syntax.Recv {  
            valid = true  
        }  
    }  
  
    if !valid {  
        check.error(clause.Comm, InvalidSelectCase, "select case must be send or receive...")  
        continue  
    }  
    ...  
}
```

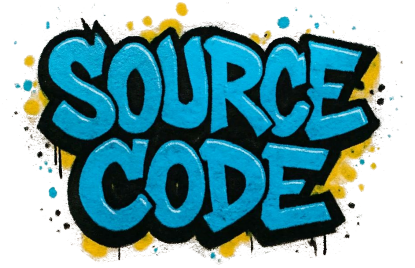

IR OPTIMIZATIONS

- ***PER CASES OPTIMIZATIONS***
 - ***ZERO CASES***
 - ***ONE CASE***
 - ***ONE CASE + DEFAULT***
 - ***N CASES***

ZERO CASES



ZERO CASES



select {}

ZERO CASES



```
if ncas == 0 {  
    return []ir.Node{mkcallstmt("block")}  
}
```

ZERO CASES

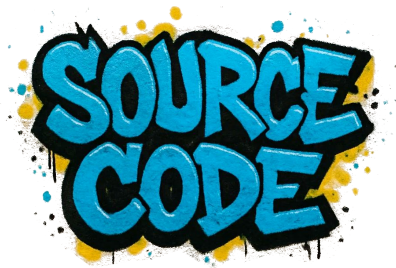


runtime.block()

ONE CASE SELECT



ONE CASE SELECT



```
select {  
  default:  
    fmt.Println("Default")  
}
```

ONE CASE SELECT



```
if ncas == 1 {  
    cas := cases[0]  
    ir.SetPos(cas)  
    l := cas.Init()  
    if cas.Comm != nil { // not default:  
        ...  
    }  
  
    l = append(l, cas.Body...)  
    l = append(l, ir.NewBranchStmt(base.Pos, ir.OBREAK, nil))  
    return l  
}
```

ONE CASE SELECT



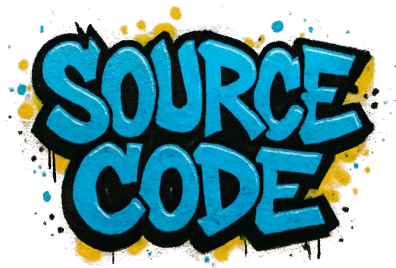
```
if ncas == 1 {  
    cas := cases[0]  
    ir.SetPos(cas)  
    l := cas.Init()  
    if cas.Comm != nil { // not default:  
        ...  
    }  
  
    l = append(l, cas.Body...)  
    l = append(l, ir.NewBranchStmt(base.Pos, ir.OBREAK, nil))  
    return l  
}
```

ONE CASE SELECT



```
fmt.Println("Default")
```

ONE CASE SELECT



```
select {  
  case msg1 := <-ch1:  
    fmt.Println("Received:", msg1)  
}
```

ONE CASE SELECT



```
n := cas.Comm
l = append(l, ir.TakeInit(n)...)
switch n.Op() {
default:
    base.Fatalf("select %v", n.Op())

case ir.OSEND:
    // already ok

case ir.OSELRECV2:
    r := n.(*ir.AssignListStmt)
    if ir.IsBlank(r.Lhs[0]) && ir.IsBlank(r.Lhs[1]) {
        n = r.Rhs[0]
        break
    }
    r.SetOp(ir.OAS2RECV)
}
```

l = append(l, n)

src/cmd/compiler/internal/walk/select.go

ONE CASE SELECT



```
n := cas.Comm
l = append(l, ir.TakeInit(n)...)
switch n.Op() {
default:
    base.Fatalf("select %v", n.Op())

case ir.OSEND:
    // already ok

case ir.OSELRECV2:
    r := n.(*ir.AssignListStmt)
    if ir.IsBlank(r.Lhs[0]) && ir.IsBlank(r.Lhs[1]) {
        n = r.Rhs[0]
        break
    }
    r.SetOp(ir.OAS2RECV)
}
```

l = append(l, n)

[src/cmd/compile/internal/walk/select.go](https://sourcegraph.com/go/src/cmd/compile/internal/walk/select.go)

ONE CASE SELECT

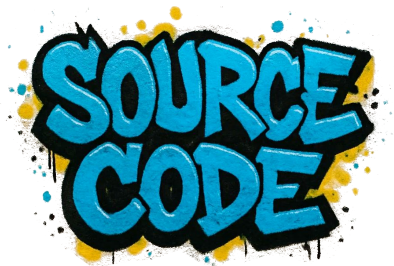


```
msg1 := <-ch1:  
fmt.Println("Received:", msg1)
```

ONE CASE + DEFAULT SELECT



ONE CASE + DEFAULT SELECT



```
select {  
  case msg := <-ch:  
    fmt.Println("Received:", msg)  
  default:  
    fmt.Println("No message received")  
}
```

ONE CASE + DEFAULT SELECT



```
if ncas == 2 && dflt != nil {  
    cas := cases[0]  
    if cas == dflt {  
        cas = cases[1]  
    }  
  
    n := cas.Comm  
    ir.SetPos(n)  
    r := ir.NewIfStmt(base.Pos, nil, nil, nil)  
    r.SetInit(cas.Init())  
    var cond ir.Node  
    switch n.Op() {  
        ...  
    }  
  
    r.Cond = typecheck.Expr(cond)  
    r.Body = cas.Body  
    r.Else = append(dflt.Init(), dflt.Body...)  
    return []ir.Node{r, ir.NewBranchStmt(base.Pos, ir.OBREAK, nil)}  
}
```

<src/cmd/compile/internal/walk/select.go>

ONE CASE + DEFAULT SELECT



```
if ncas == 2 && dflt != nil {  
    cas := cases[0]  
    if cas == dflt {  
        cas = cases[1]  
    }  
  
    n := cas.Comm  
    ir.SetPos(n)  
    r := ir.NewIfStmt(base.Pos, nil, nil, nil)  
    r.SetInit(cas.Init())  
    var cond ir.Node  
    switch n.Op() {  
        ...  
    }  
  
    r.Cond = typecheck.Expr(cond)  
    r.Body = cas.Body  
    r.Else = append(dflt.Init(), dflt.Body...)  
    return []ir.Node{r, ir.NewBranchStmt(base.Pos, ir.OBREAK, nil)}  
}
```

<src/cmd/compile/internal/walk/select.go>

ONE CASE + DEFAULT SELECT



```
switch n.Op() {  
    ...  
  
    case ir.OSELRECV2:  
        n := n.(*ir.AssignListStmt)  
        recv := n.Rhs[0].(*ir.UnaryExpr)  
        ch := recv.X  
        elem := n.Lhs[0]  
        if ir.IsBlank(elem) {  
            elem = typecheck.NodNil()  
        }  
        cond = typecheck.TempAt(base.Pos, ir.CurFunc, types.Types[types.TBOOL])  
        fn := chanfn("selectnbrecv", 2, ch.Type())  
        call := mkcall1(fn, fn.Type().ResultsTuple(), r.PtrInit(), elem, ch)  
        as := ir.NewAssignListStmt(r.Pos(), ir.OAS2, []ir.Node{cond, n.Lhs[1]}, []ir.Node{call})  
        r.PtrInit().Append(typecheck.Stmt(as))  
}
```

<src/cmd/compile/internal/walk/select.go>

ONE CASE + DEFAULT SELECT



```
switch n.Op() {
```

```
...
```

```
case ir.OSELRECV2:
```

```
    n := n.(*ir.AssignListStmt)
```

```
    recv := n.Rhs[0].(*ir.UnaryExpr)
```

```
    ch := recv.X
```

```
    elem := n.Lhs[0]
```

```
    if ir.IsBlank(elem) {
```

```
        elem = typecheck.NodNil()
```

```
    }
```

```
    cond = typecheck.TempAt(base.Pos, ir.CurFunc, types.Types[types.TBOOL])
```

```
    fn := chanfn("selectnbrecv", 2, ch.Type())
```

```
    call := mkcall1(fn, fn.Type().ResultsTuple(), r.PtrInit(), elem, ch)
```

```
    as := ir.NewAssignListStmt(r.Pos(), ir.OAS2, []ir.Node{cond, n.Lhs[1]}, []ir.Node{call})
```

```
    r.PtrInit().Append(typecheck.Stmt(as))
```

```
}
```

<src/cmd/compile/internal/walk/select.go>

ONE CASE + DEFAULT SELECT

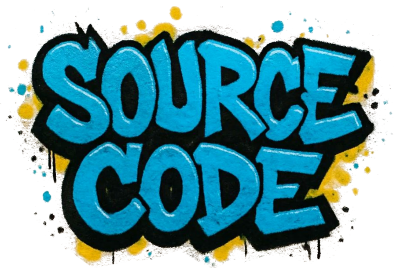


```
if runtime.selectnbrecv(ch) {  
    fmt.Println("Received:", msg)  
} else {  
    fmt.Println("No message received")  
}
```

N CASES



N CASES



```
select {  
  case msg1 := <-ch1:  
    fmt.Println("Received:", msg1)  
  case msg2 := <-ch2:  
    fmt.Println("Received:", msg2)  
}
```

N CASES



```
fn := typecheck.LookupRuntime("selectgo")
var fnInit ir.Nodes
r.Rhs = []ir.Node{mkcall1(
    fn,
    fn.Type().ResultsTuple(),
    &fnInit,
    bytePtrToIndex(selv, 0),
    bytePtrToIndex(order, 0),
    pc0,
    ir.NewInt(base.Pos, int64(nsends)),
    ir.NewInt(base.Pos, int64(nrecvs)),
    ir.NewBool(base.Pos, dflt == nil))
}
```

src/cmd/compile/internal/walk/select.go

N CASES



`runtime.selectgo(...)`

SELECT AND THE RUNTIME



SELECT AND THE RUNTIME

- ***SELECTNBSEND***
- ***SELECTNBRECV***
- ***SELECTGO***

SELECT AND THE RUNTIME



```
func selectnbsend(c *hchan, elem unsafe.Pointer) (selected bool) {  
    return chansend(c, elem, false, sys.GetCallerPC())  
}
```

```
func selectnbrecv(elem unsafe.Pointer, c *hchan) (selected, received bool) {  
    return chanrecv(c, elem, false)  
}
```

SELECT AND THE RUNTIME



```
func selectnbsend(c *hchan, elem unsafe.Pointer) (selected bool) {  
    return chansend(c, elem, false, sys.GetCallerPC())  
}
```

```
func selectnbrekv(elem unsafe.Pointer, c *hchan) (selected, received bool) {  
    return chanrecv(c, elem, false)  
}
```

SELECTNBRECV



```
if !block && empty(c) {  
    if atomic.Load(&c.closed) == 0 {  
        return  
    }  
    if empty(c) {  
        if ep != nil {  
            typedmemclr(c.elemtype, ep)  
        }  
        return true, false  
    }  
}
```

SELECTNBRECV



```
if !block && empty(c) {  
    if atomic.Load(&c.closed) == 0 {  
        return  
    }  
    if empty(c) {  
        if ep != nil {  
            typedmemclr(c.elemtype, ep)  
        }  
        return true, false  
    }  
}
```

SELECTNBSEND



```
if !block && c.closed == 0 && full(c) {  
    return false  
}
```

... // checking listeners and buffer

```
if !block {  
    unlock(&c.lock)  
    return false  
}
```

SELECTNBSEND



```
if !block && c.closed == 0 && full(c) {  
    return false  
}
```

... // checking listeners and buffer

```
if !block {  
    unlock(&c.lock)  
    return false  
}
```

SELECTNBSEND



```
if !block && c.closed == 0 && full(c) {  
    return false  
}
```

... // checking listeners and buffer

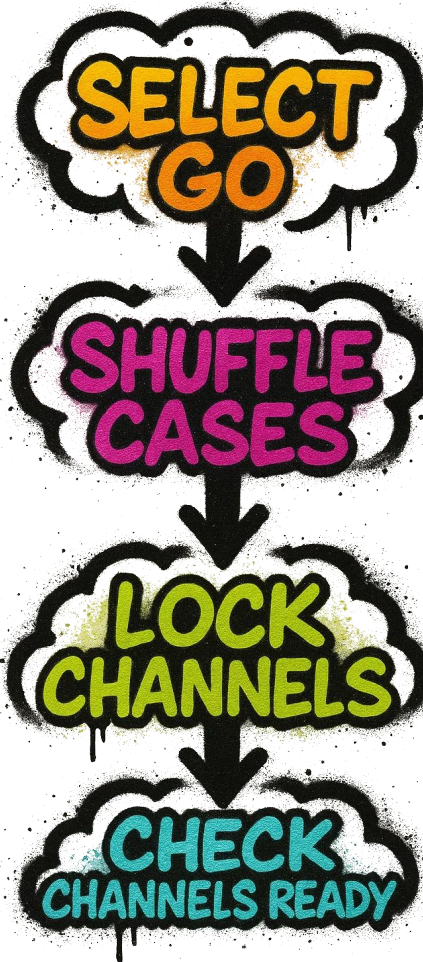
```
if !block {  
    unlock(&c.lock)  
    return false  
}
```

SELECT AND THE RUNTIME

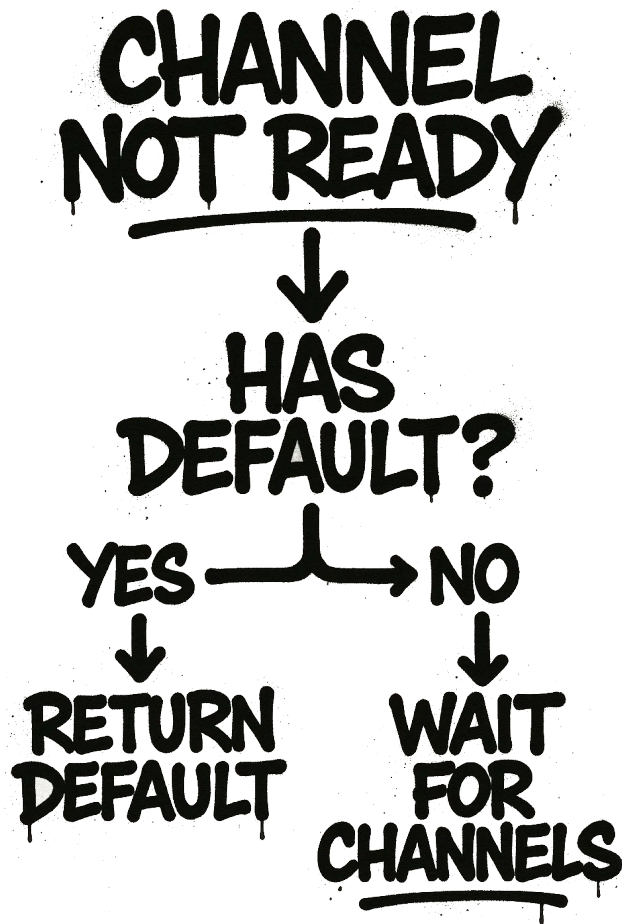


```
func selectgo(  
    cas0 *scase,  
    order0 *uint16,  
    pc0 *uintptr,  
    nsends int,  
    nrecvs int,  
    block bool  
) (int, bool) {
```


SELECTGO



SELECTGO



SELECTGO

**CHANNEL
READY**



**WAKE THE
GOROUTINE**



DESUBSCRIBE



**UNLOCK
CHANNELS**



**RETURN CHOSEN
CHANNEL**

FAIRNESS



FAIRNESS

```
norder := 0
for i := range scases {
    cas := &scases[i]

    // Omit cases without channels from the poll and lock orders.
    if cas.c == nil {
        cas.elem = nil // allow GC
        continue
    }

    if cas.c.timer != nil {
        cas.c.timer.maybeRunChan(cas.c)
    }

    j := cheaprandn(uint32(norder + 1))
    pollorder[norder] = pollorder[j]
    pollorder[j] = uint16(i)
    norder++
}
pollorder = pollorder[:norder]
```



FAIRNESS

```
norder := 0
for i := range scases {
    cas := &scases[i]

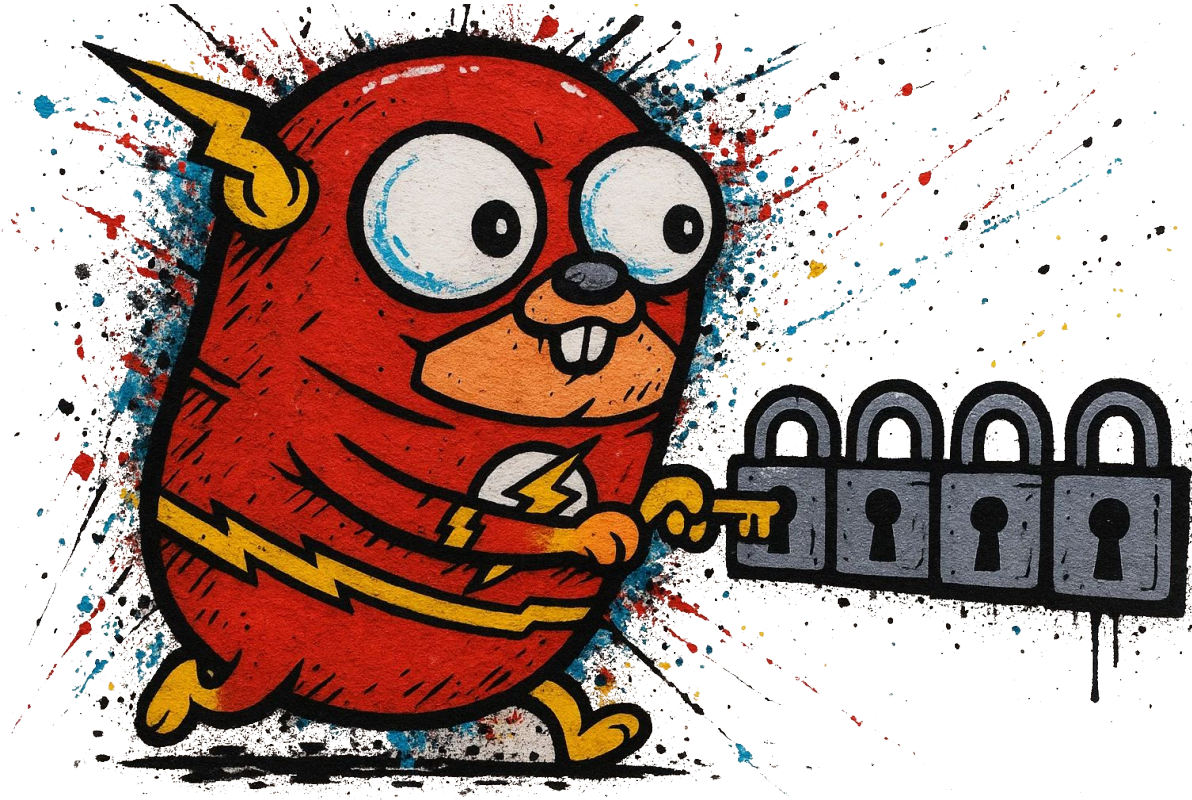
    // Omit cases without channels from the poll and lock orders.
    if cas.c == nil {
        cas.elem = nil // allow GC
        continue
    }

    if cas.c.timer != nil {
        cas.c.timer.maybeRunChan(cas.c)
    }

    j := cheaprandn(uint32(norder + 1))
    pollorder[norder] = pollorder[j]
    pollorder[j] = uint16(i)
    norder++
}
pollorder = pollorder[:norder]
```



SORTED LOCK



SORTED LOCK



```
for i := range lockorder {  
    j := i  
    // Start with the pollorder to permute cases on the same channel.  
    c := scases[pollorder[i]].c  
    for j > 0 && scases[lockorder[(j-1)/2]].c.sortkey() < c.sortkey() {  
        k := (j - 1) / 2  
        lockorder[j] = lockorder[k]  
        j = k  
    }  
    lockorder[j] = pollorder[i]  
}
```


SORTED LOCK

```
for i := len(lockorder) - 1; i >= 0; i-- {
    o := lockorder[i]
    c := scases[o].c
    lockorder[i] = lockorder[0]
    j := 0
    for {
        k := j*2 + 1
        if k >= i {
            break
        }
        if k+1 < i && scases[lockorder[k]].c.sortkey() < scases[lockorder[k+1]].c.sortkey() {
            k++
        }
        if c.sortkey() < scases[lockorder[k]].c.sortkey() {
            lockorder[j] = lockorder[k]
            j = k
            continue
        }
        break
    }
    lockorder[j] = o
}
```



SORTED LOCK



```
// lock all the channels involved in the select  
sellock(scases, lockorder)
```

READY TO FLY



READY TO FLY



```
for _, casei := range pollorder {  
    ...  
    if casi >= nsends {  
        sg = c.sendq.dequeue()  
        if sg != nil { goto recv }  
        if c.qcount > 0 { goto bufrecv }  
        if c.closed != 0 { goto rclose }  
    } else {  
        if c.closed != 0 { goto sclose }  
        sg = c.recvq.dequeue()  
        if sg != nil { goto send }  
        if c.qcount < c.dataqsiz { goto bufsend }  
    }  
}
```

READY TO FLY



```
for _, casei := range pollorder {  
    ...  
    if casi >= nsends {  
        sg = c.sendq.dequeue()  
        if sg != nil { goto recv }  
        if c.qcount > 0 { goto bufrecv }  
        if c.closed != 0 { goto rclose }  
    } else {  
        if c.closed != 0 { goto sclose }  
        sg = c.recvq.dequeue()  
        if sg != nil { goto send }  
        if c.qcount < c.dataqsiz { goto bufsend }  
    }  
}
```

NOT BLOCKING



SORTED LOCK



```
if !block {  
    selunlock(scases, lockorder)  
    casi = -1  
    goto retc  
}
```


GO TO SLEEP



SORTED LOCK



```
for _, casei := range lockorder {  
    ...  
  
    if casi < nsends {  
        c.sendq.enqueue(sg)  
    } else {  
        c.recvq.enqueue(sg)  
    }  
  
    ...  
}  
...  
gopark(selparkcommit, nil, waitReason, traceBlockSelect, 1)
```

SORTED LOCK



```
for _, casei := range lockorder {  
    ...  
  
    if casi < nsends {  
        c.sendq.enqueue(sg)  
    } else {  
        c.recvq.enqueue(sg)  
    }  
  
    ...  
}  
...  
gopark(selparkcommit, nil, waitReason, traceBlockSelect, 1)
```

SORTED LOCK



```
for _, casei := range lockorder {  
    ...  
  
    if casi < nsends {  
        c.sendq.enqueue(sg)  
    } else {  
        c.recvq.enqueue(sg)  
    }  
  
    ...  
}  
...  
gopark(sel parkcommit, nil, waitReason, traceBlockSelect, 1)
```

WAKE UP



WAKE UP

```
for _, casei := range lockorder {  
    ...  
    if sg == sglist {  
        // sg has already been dequeued by the G that woke us up.  
        casi = int(casei)  
        cas = k  
        caseSuccess = sglist.success  
        if sglist.releasetime > 0 {  
            caseReleaseTime = sglist.releasetime  
        }  
    } else {  
        c = k.c  
        if int(casei) < nsends {  
            c.sendq.dequeueSudoG(sglist)  
        } else {  
            c.recvq.dequeueSudoG(sglist)  
        }  
    }  
    ...  
}
```



WAKE UP

```
for _, casei := range lockorder {  
    ...  
    if sg == sglist {  
        // sg has already been dequeued by the G that woke us up.  
        casi = int(casei)  
        cas = k  
        caseSuccess = sglist.success  
        if sglist.releasetime > 0 {  
            caseReleaseTime = sglist.releasetime  
        }  
    } else {  
        c = k.c  
        if int(casei) < nsends {  
            c.sendq.dequeueSudoG(sglist)  
        } else {  
            c.recvq.dequeueSudoG(sglist)  
        }  
    }  
    ...  
}
```



WAKE UP

```
for _, casei := range lockorder {  
    ...  
    if sg == sglist {  
        // sg has already been dequeued by the G that woke us up.  
        casi = int(casei)  
        cas = k  
        caseSuccess = sglist.success  
        if sglist.releasetime > 0 {  
            caseReleaseTime = sglist.releasetime  
        }  
    } else {  
        c = k.c  
        if int(casei) < nsends {  
            c.sendq.dequeueSudoG(sglist)  
        } else {  
            c.recvq.dequeueSudoG(sglist)  
        }  
    }  
    ...  
}
```



DOING THE WORK



DOING THE WORK

```
chosen, recvOK := runtime.selectgo(...)
```

```
if chosen < 0 {  
    <DEFAULT BODY>  
    goto end  
}
```

```
if chosen == 0 {  
    <CASE 0 BODY>  
    goto end  
}
```

```
if chosen == 1 {  
    <CASE 1 BODY>  
    goto end  
}
```

```
}
```

```
...
```

```
end:
```

```
// Continue with code after select
```



SUMMARY



REFERENCES



- **THE GO SOURCE CODE**
- **[HTTPS://GO.DEV/REF/SPEC#SELECT_STATEMENTS](https://go.dev/ref/spec#SELECT_STATEMENTS)**
- **[HTTPS://INTERNALS-FOR-INTERNS.COM/POSTS/GO-RUNTIME-SELECT/](https://internals-for-interns.com/posts/go-runtime-select/)**
- **ARNE CLAUS - CONCURRENCY PATTERNS IN GO:
[HTTPS://WWW.YOUTUBE.COM/WATCH?V=RDRA23K70CU](https://www.youtube.com/watch?v=RDRA23K70CU)**

LET'S CONNECT



<https://bit.ly/jesus-espino-links>

